Symbiosis Institute of Technology, Pune

Faculty of Engineering
CSE- Academic Year 2026-27
Java Programming – Lab Batch 2024-28

Lab Assignment No:- 28	
Name of Student	Abishek Yadav
PRN No.	25070122014
Batch	A1
Class	CSE
Academic Year & Semester	2026-2027/3RD
Date of Submission	18/10/2026
Title of Assignment:	Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.
Theory:	Remote Method Invocation (RMI) is a Java technology that enables an object running on one Java Virtual Machine (JVM) to invoke methods of an object running on another JVM located on a different machine. RMI simplifies distributed application development by allowing remote objects to behave like local objects. It consists of remote interfaces, remote object implementations, the RMI registry, and client-server communication. The registry maintains references to remote objects, allowing clients to locate and invoke them. RMI provides platform-independent communication, object serialization, and automatic network handling, making it suitable for distributed enterprise applications.
Practice Questions	1. Create a remote calculator service using Java RMI.

CODE:

```java
// Calculator.java
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    int divide(int a, int b) throws RemoteException;
}

// CalculatorImpl.java
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject
        implements Calculator {
    public CalculatorImpl() throws RemoteException {
        super();
    }
    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
    public int subtract(int a, int b) throws RemoteException {
        return a - b;
    }
    public int multiply(int a, int b) throws RemoteException {
        return a * b;
    }
    public int divide(int a, int b) throws RemoteException {
        return a / b;
    }
}
```

CODE:

```java
// CalculatorServer.java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
```

```
    public static void main(String[] args) {
        try {
            CalculatorImpl calculator = new CalculatorImpl();
            Registry registry =
LocateRegistry.createRegistry(2000);
            registry.rebind("CalculatorService", calculator);
            System.out.println("Calculator RMI Server started.");
            System.out.println("Waiting for client...");
        } catch (Exception e) {
            System.out.println("Server Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>javac Calculator.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>javac CalculatorImpl.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>javac CalculatorServer.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>javac CalculatorClient.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>java CalculatorServer
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

Calculator RMI Server started.
Waiting for client...
```

CODE:

```
// CalculatorClient.java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry =
                LocateRegistry.getRegistry("localhost", 2000);
            Calculator calculator =
                (Calculator) registry.lookup("CalculatorService");
            int a = 20;
            int b = 10;
            System.out.println("Addition = "
                + calculator.add(a, b));
            System.out.println("Subtraction = "
                + calculator.subtract(a, b));
            System.out.println("Multiplication = "
                + calculator.multiply(a, b));
            System.out.println("Division = "
                + calculator.divide(a, b));
        } catch (Exception e) {
            System.out.println("Client Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>java CalculatorClient
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2

C:\Users\Bina Computer\OneDrive\Documents\Java>
```

QUESTION

Create a Java application using RMI where the server converts temperature values between Celsius and Fahrenheit. The client sends input temperature and receives converted output.

CODE:

```
// Temperature.java
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Temperature extends Remote {
    double celsiusToFahrenheit(double celsius)
            throws RemoteException;
    double fahrenheitToCelsius(double fahrenheit)
            throws RemoteException;
}
```

```java
// TemperatureImpl.java
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class TemperatureImpl extends UnicastRemoteObject
        implements Temperature {
    public TemperatureImpl() throws RemoteException {
        super();
    }
    public double celsiusToFahrenheit(double celsius)
            throws RemoteException {
        return (celsius * 9 / 5) + 32;
    }
    public double fahrenheitToCelsius(double fahrenheit)
            throws RemoteException {
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

CODE:

```java
// TemperatureServer.java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class TemperatureServer {
    public static void main(String[] args) {
        try {
            TemperatureImpl temperature = new TemperatureImpl();
            Registry registry =
                LocateRegistry.createRegistry(1099);
            registry.rebind("TemperatureService", temperature);
            System.out.println("Temperature RMI Server started.");
            System.out.println("Waiting for client...");
        } catch (Exception e) {
            System.out.println("Server Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>javac Temperature.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>javac TemperatureImpl.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>javac TemperatureServer.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>javac TemperatureClient.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

C:\Users\Bina Computer\OneDrive\Documents\Java>java TemperatureServer
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

Temperature RMI Server started.
Waiting for client...
```

CODE:

```java
// TemperatureClient.java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class TemperatureClient {
    public static void main(String[] args) {
        try {
            Registry registry =
                LocateRegistry.getRegistry("localhost", 1099);
            Temperature temperature =
                (Temperature)
registry.lookup("TemperatureService");
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter temperature in Celsius: ");
            double celsius = sc.nextDouble();
            double fahrenheit =
                temperature.celsiusToFahrenheit(celsius);
            System.out.println(
                "Temperature in Fahrenheit = "
                + fahrenheit
            );
            System.out.print("Enter temperature in Fahrenheit: ");
            double fahrenheitInput = sc.nextDouble();
            double celsiusOutput =
                temperature.fahrenheitToCelsius(fahrenheitInput);
            System.out.println(
                "Temperature in Celsius = "
```

```
                + celsiusOutput
            );
            sc.close();
        } catch (Exception e) {
            System.out.println("Client Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:

```
C:\Users\Bina Computer\OneDrive\Documents\Java>java TemperatureClient
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

Enter temperature in Celsius: 25
Temperature in Fahrenheit = 77.0
Enter temperature in Fahrenheit: 98.6
Temperature in Celsius = 37.0

C:\Users\Bina Computer\OneDrive\Documents\Java>
```

Conclusion	The Java RMI applications successfully demonstrate remote communication between a client and server. The first application performs calculator operations remotely, while the second converts temperatures between Celsius and Fahrenheit. The programs demonstrate the use of remote interfaces, remote objects, the RMI registry, client-side stubs, and remote method calls.